

ENTREGA FINAL

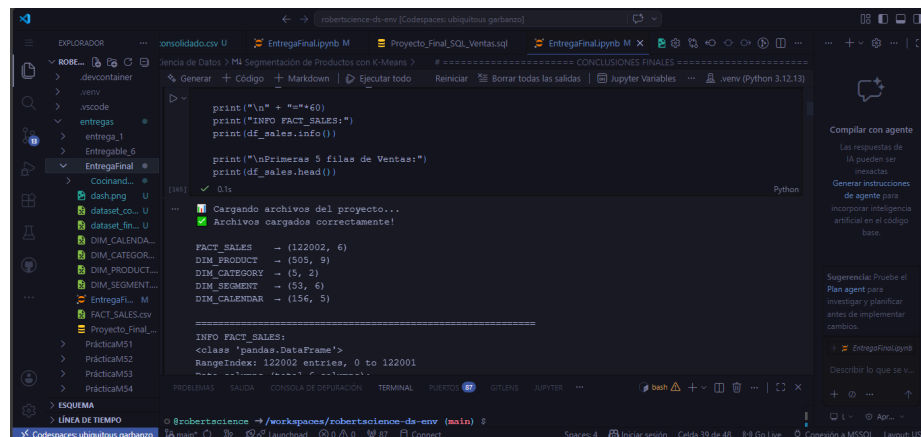
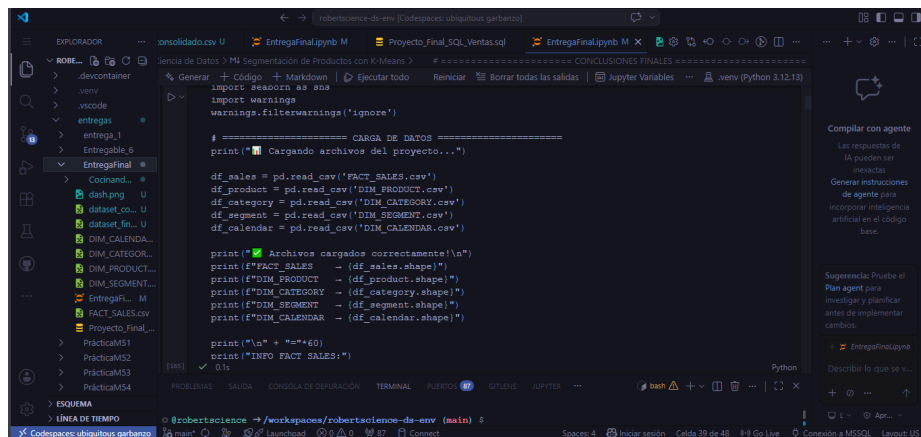
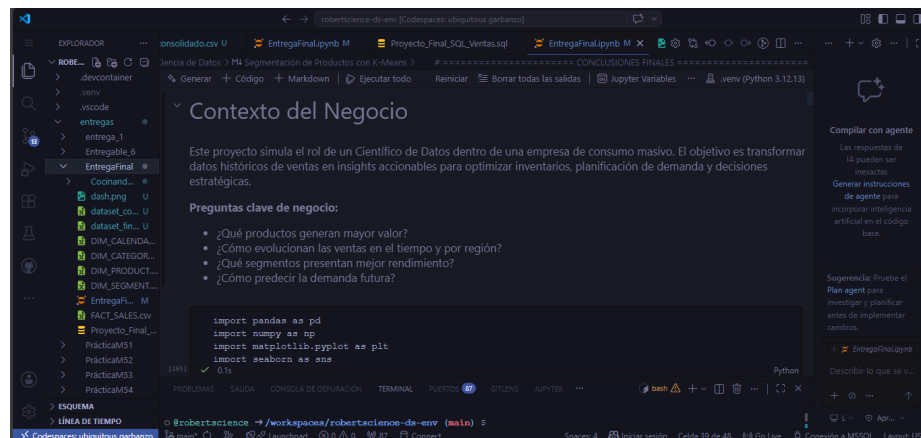
Proyecto de Ciencia de Datos

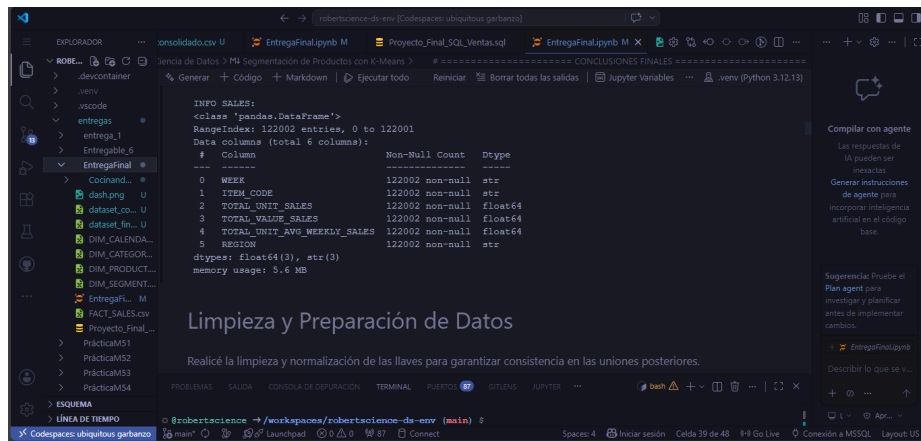
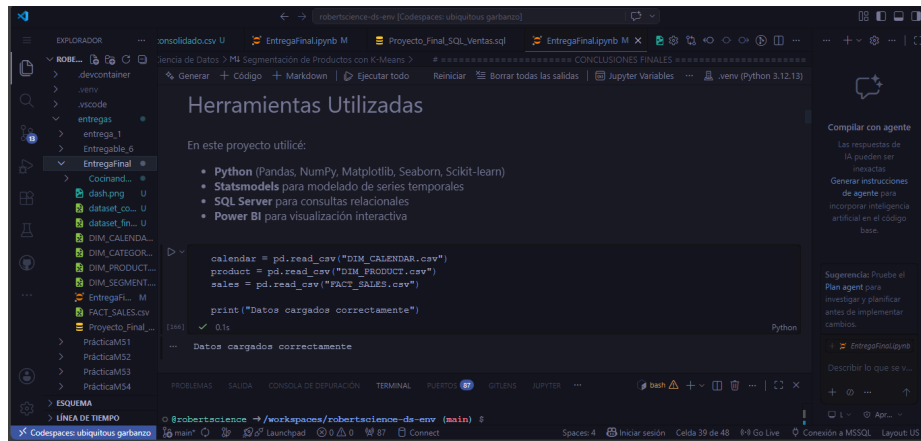
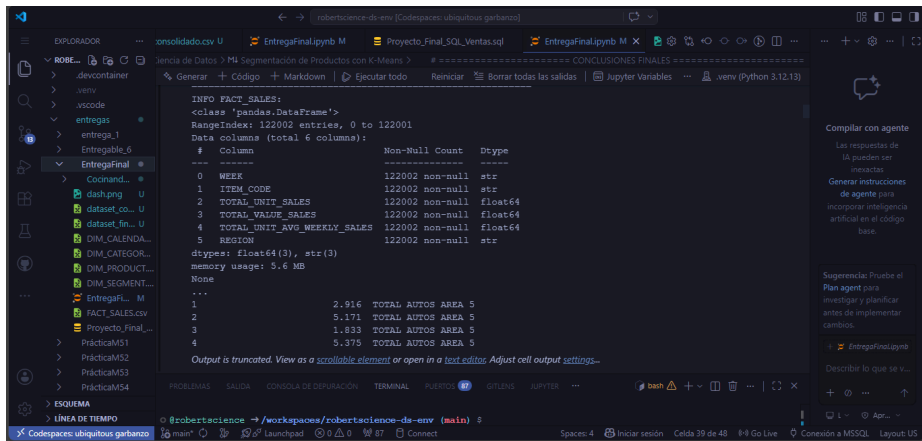
Análisis de Ventas y Pronóstico de Demanda

En este proyecto actué como Científico de Datos en una empresa de productos de consumo masivo. El objetivo principal fue analizar el comportamiento histórico de las ventas y construir un modelo predictivo que permitiera estimar la demanda futura de productos clave como Vanish y Lysol.

Trabajé con un modelo dimensional de datos, realizando carga, limpieza, consolidación, análisis exploratorio, segmentación mediante K-Means y modelado de series temporales con ARIMA.

Para ello utilicé: Python (Pandas, NumPy, Matplotlib, Seaborn, Scikit-learn, Statsmodels), SQL Server y Power BI.





EntregaFinal.py M

```
print(df_product.head())
```

MANUFACTURER	BRAND	ITEM	
0	INDS. ALLEN	CLORALEX	0000075000592
1	INDS. ALLEN	CLORALEX	0000075000609
2	INDS. ALLEN	CLORALEX	0000075000615
3	INDS. ALLEN	CLORALEX	0000075000622
4	INDS. ALLEN	CLORALEX	0000075000639

ITEM DESCRIPTION	CATEGORY	FORMAT	
0	CLORALEX EL RENDIDOR BOT. PLAST.	250ML NAL.	000...
1	CLORALEX EL RENDIDOR BOT. PLAST.	500ML NAL.	000...
2	CLORALEX EL RENDIDOR BOT. PLAST.	500ML NAL.	000...
3	CLORALEX EL RENDIDOR BOT. PLAST.	2000ML NAL.	000...
4	CLORALEX EL RENDIDOR BOT. PLAST.	3750ML NAL.	000...

ATTR1	ATTR2	ATTR3	
0	CLORO	CLORO	NO DEFINIDO

EntregaFinal.py M

```
print(df_product.columns)
```

```
Index(['MANUFACTURER', 'BRAND', 'ITEM', 'ITEM DESCRIPTION', 'CATEGORY', 'FORMAT', 'ATTR1', 'ATTR2', 'ATTR3'], dtype='str')
```

```
df_product.columns = df_product.columns.str.strip().str.upper()
```

EntregaFinal.py M

```
Index(['MANUFACTURER', 'BRAND', 'ITEM', 'ITEM DESCRIPTION', 'CATEGORY', 'FORMAT', 'ATTR1', 'ATTR2', 'ATTR3'], dtype='str')
```

```
===== NORMALIZACIÓN DE COLUMNAS =====
for df_name, df in [{"Sales": df_sales}, {"Product": df_product}, {"Category": df_category}, {"Segment": df_segment}, {"Calendar": df_calendar}]:
    df.columns = df.columns.str.strip().str.upper()
    print(f"✓ (df_name) columnas normalized")

print("\nColumnas finales:")
print("Sales:", df_sales.columns.tolist())
print("Product:", df_product.columns.tolist())

===== LIMPIEZA DE LLAVES =====
df_sales["ITEM_CLEAN"] = df_sales["ITEM_CODE"].astype(str).str.extract(r'(\d+)')
df_sales["ITEM_CLEAN"] = df_sales["ITEM_CLEAN"].str.zfill(13)
```

EntregaFinal.py M

```
df_product["ITEM_CLEAN"] = df_product["ITEM"].astype(str).str.extract(r'(\d+)')
df_product["ITEM_CLEAN"] = df_product["ITEM_CLEAN"].str.zfill(13)
```

```
print("✓ Llaves limpiadas correctamente")
print("Ejemplo ITEM_CLEAN Sales:", df_sales["ITEM_CLEAN"].head())
print("Ejemplo ITEM_CLEAN Product:", df_product["ITEM_CLEAN"].head())
```

```
===== CONSOLIDACIÓN (MERGE) =====
df_merged = df_sales.merge(df_product, left_on="ITEM_CLEAN", right_on="ITEM_CLEAN", how="left")

df_calendar["WEEK"] = df_calendar["WEEK"].astype(str)
df_merged = df_merged.merge(df_calendar, on="WEEK", how="left")

print("\n" + "="*70)
print("✓ DATASET CONSOLIDADO")
print("Shape final:", df_merged.shape)
print("\nValores nulos después del merge:")
print(df_merged.isnull().sum())

print("\nPrimeras 5 filas:")
print(df_merged.head())
```

EntregaFinal.py

```
print("\nSe ingresaron 5 files:")
print(df_merged.head())
```

171 ✓ 0.4s Python

✓ Sales columns normalized
✓ Product columns normalized
✓ Category columns normalized
✓ Segment columns normalized
✓ Calendar columns normalized

Columnas finales:
Sales: ['WEEK', 'ITEM_CODE', 'TOTAL_UNIT_SALES', 'TOTAL_VALUE_SALES', 'TOTAL_UNIT_AVG_WEEKLY_SALES', 'REGION', 'Product: ['MANUFACTURER', 'BRAND', 'ITEM', 'ITEM_DESCRIPTION', 'CATEGORY', 'FORMAT', 'Attr1', 'Attr2', 'Attr3']

✓ Llaves limpiadas correctamente
Ejemplo ITEM_CLEAN Sales: 0 7501058792808
1 7501058715883
2 7702626213774
3 7501058716422
4 7501058784353

Name: ITEM_CLEAN, dtype: str
Ejemplo ITEM_CLEAN Product: 0 0000075000592
1 0000075000600
2 0000075000615
3 0000075000622
4 0000075000639

Name: ITEM_CLEAN, dtype: str

Output is truncated. View as a [scrollable element](#) or open in a [text editor](#). Adjust cell output settings...

EntregaFinal.py

```
3 7501058716422
4 7501058784353
Name: ITEM_CLEAN, dtype: str
Ejemplo ITEM_CLEAN Product: 0 0000075000592
1 0000075000600
2 0000075000615
3 0000075000622
4 0000075000639
Name: ITEM_CLEAN, dtype: str
```

1 SAFE BLEACH FABRIC TREATMENT ROSA 2022 8 34 2022-08-28
2 SAFE BLEACH FABRIC TREATMENT ROSA 2022 8 34 2022-08-28
3 SAFE BLEACH FABRIC TREATMENT ROSA 2022 8 34 2022-08-28
4 SAFE BLEACH FABRIC TREATMENT ROSA 2022 8 34 2022-08-28

Output is truncated. View as a [scrollable element](#) or open in a [text editor](#). Adjust cell output settings...

EntregaFinal.py

```
##### MERGE FINAL #####
print("\n Iniciando consolidación...")

sales_product = df_sales.merge(df_product, left_on="ITEM_CLEAN", right_on="ITEM_CLEAN", how="left")

print("✓ Merge Sales + Product - Shape:", sales_product.shape)

df_calendar["WEEK"] = df_calendar["WEEK"].astype(str).strip()
df = sales_product.merge(df_calendar, on="WEEK", how="left")

print("\n ¡DATASET CONSOLIDADO EXITOSAMENTE!")
```

171 ✓ 1.5s Python

Integré las tablas dimensionales con la tabla de hechos para crear un dataset unificado.

EntregaFinal.py

```
df = sales_product.merge(df_calendar, on="WEEK", how="left")

print("\n ¡DATASET CONSOLIDADO EXITOSAMENTE!")
print("Shape final del dataset:", df.shape)
print("Columnas finales:", df.columns.tolist())

print("\nNulos en el dataset final:")
print(df.isnull().sum())

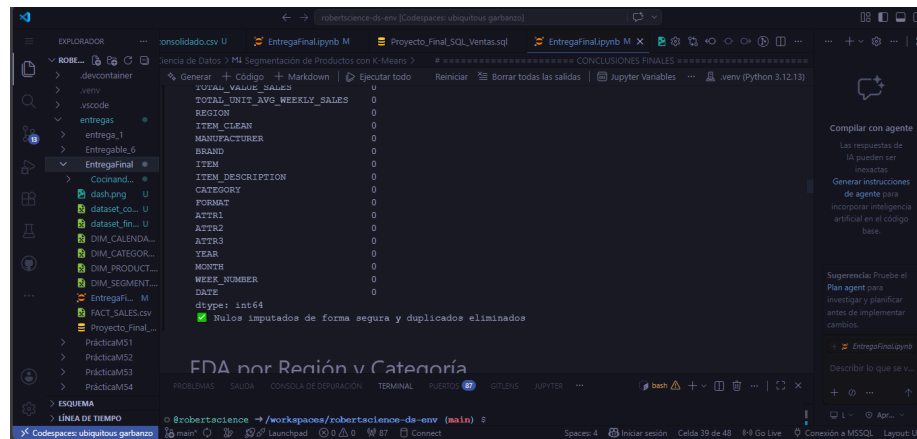
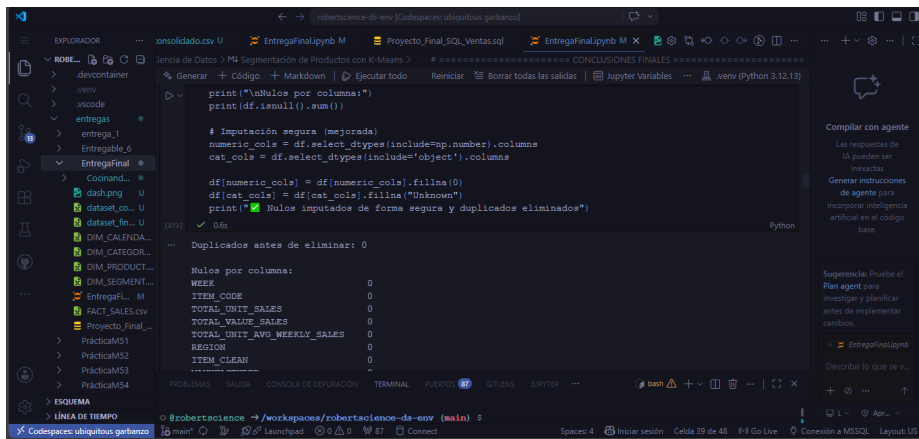
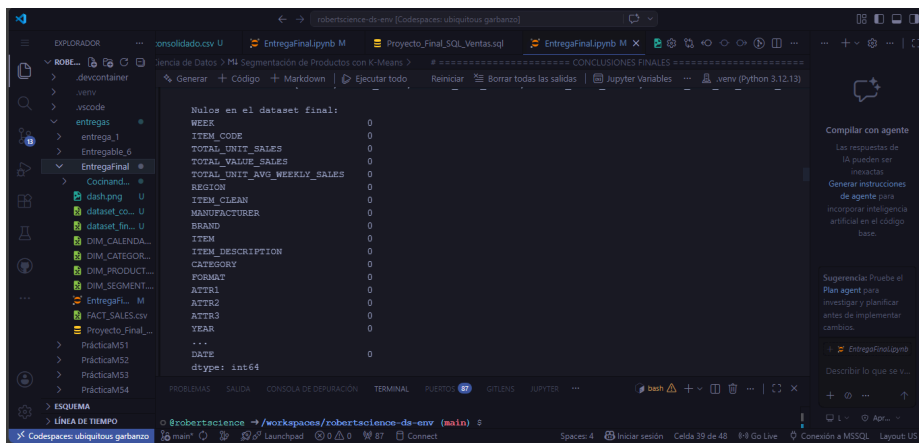
df.to_csv("dataset_consolidado.csv", index=False)
print("✓ Archivo guardado como: dataset_consolidado.csv")
```

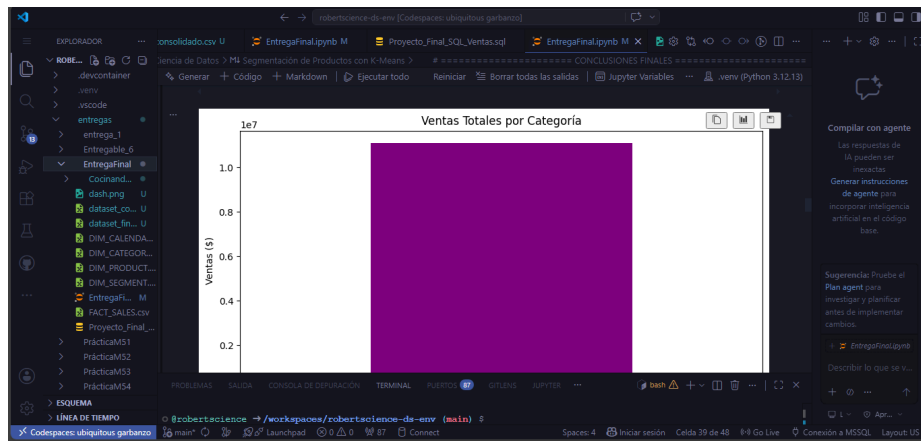
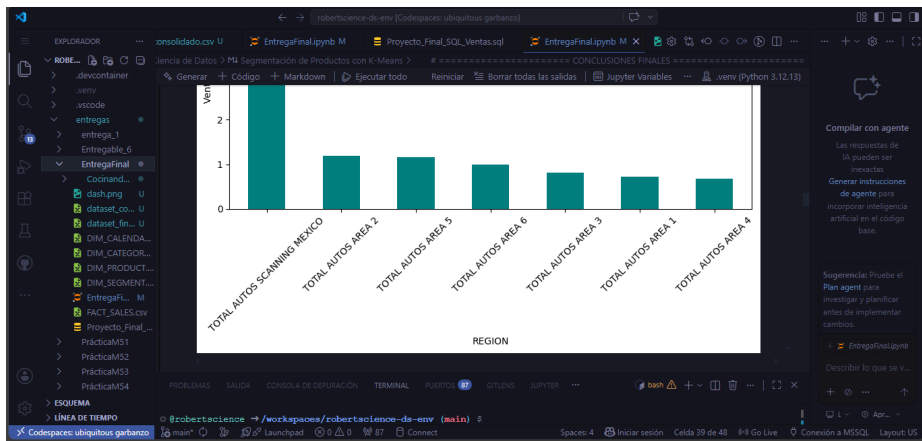
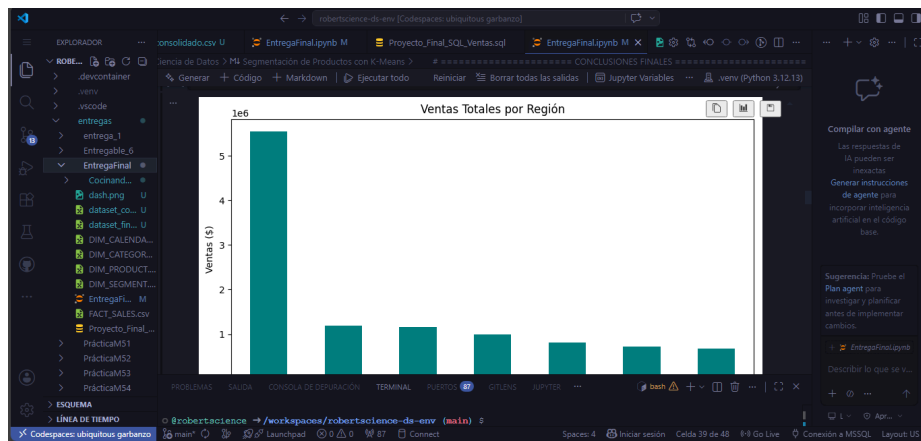
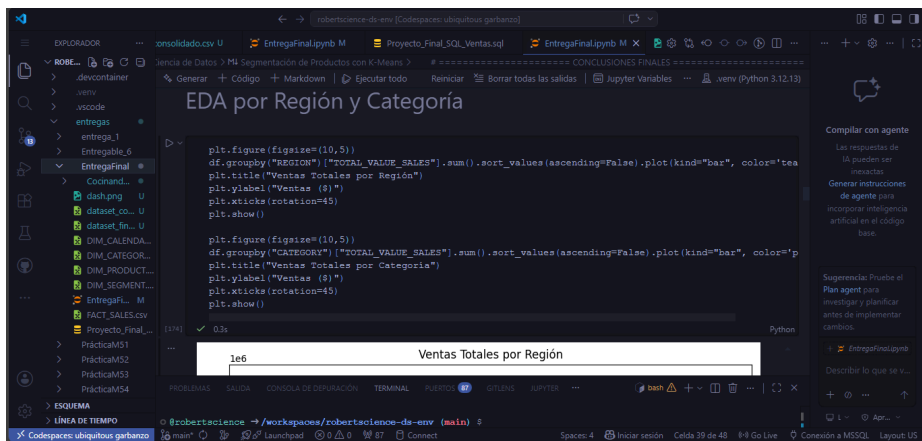
171 ✓ 1.5s Python

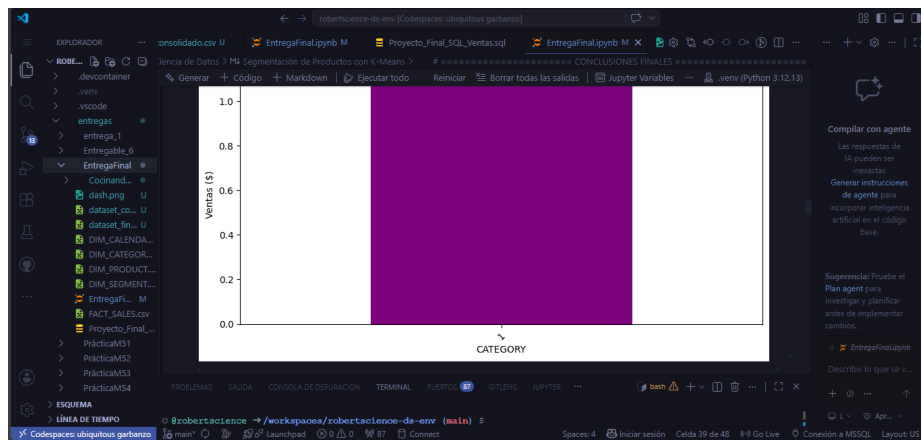
✓ Iniciando consolidación...
✓ Merge Sales + Product - Shape: (130338, 16)

¡DATASET CONSOLIDADO EXITOSAMENTE!
Shape final del dataset: (130338, 20)
Columnas finales: ['WEEK', 'ITEM_CODE', 'TOTAL_UNIT_SALES', 'TOTAL_VALUE_SALES', 'TOTAL_UNIT_AVG_WEEKLY_SALE', 'ITEM_CODE']

Nulos en el dataset final:
WEEK 0
ITEM_CODE 0







The screenshot shows a Jupyter Notebook environment. The main area displays a Python script for time series analysis. The script includes the following code:

```
# Convertir DATE a datetime
df["DATE"] = pd.to_datetime(df["DATE"])

print("Buscando productos Vanish y Lysol...")

mask = (
    df["BRAND"].astype(str).str.upper().str.contains("VANISH|LYSOL", na=False) |
    df["ITEM_DESCRIPTION"].astype(str).str.upper().str.contains("VANISH|LYSOL", na=False)
)

forecast_df = df[mask].copy()

print(f"Registros encontrados de Vanish/Lysol: {len(forecast_df)}")

ts = forecast_df.groupby("DATE")["TOTAL_VALUE_SALES"].sum().reset_index()
ts = ts.sort_values("DATE")
```

The interface also shows a file explorer on the left with files like 'entregas_1', 'Entregable_6', and 'EntregaFinal'. The bottom terminal shows the command 'bash' and the file path '/workspaces/robertaciencia-de-emv (main)'. The right sidebar contains options to 'Compile with agent' and 'Describe to me what is in this code'.

The screenshot shows a JupyterLab interface with a Jupyter Notebook open. The notebook contains the following code and output:

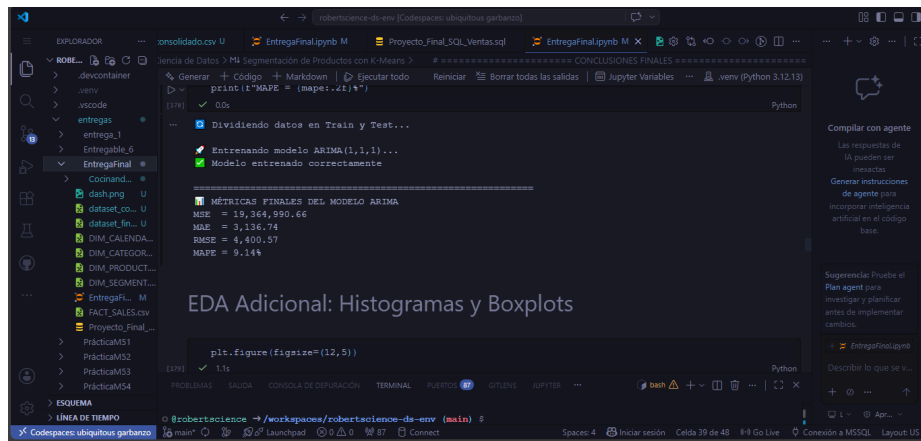
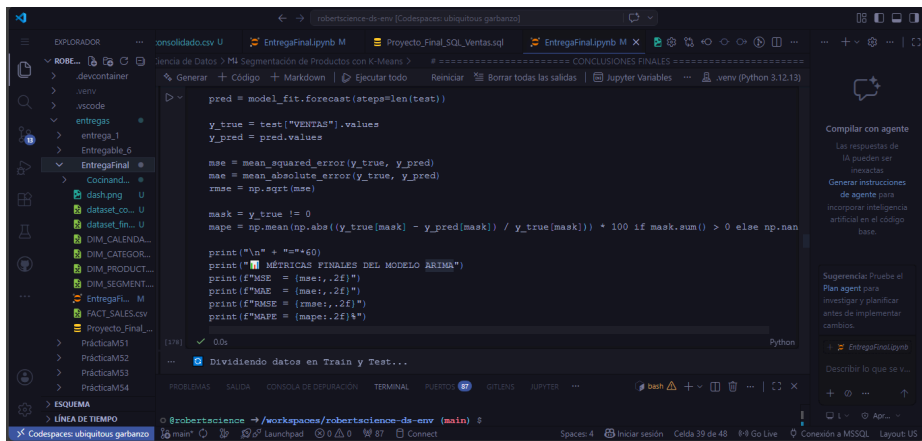
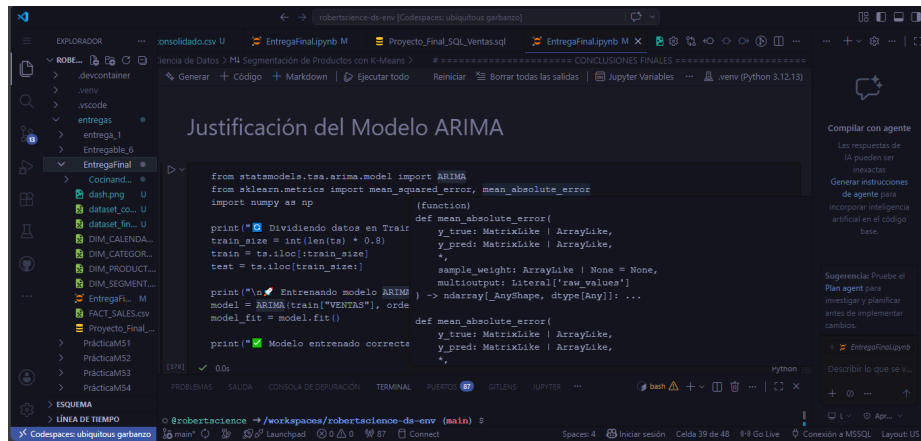
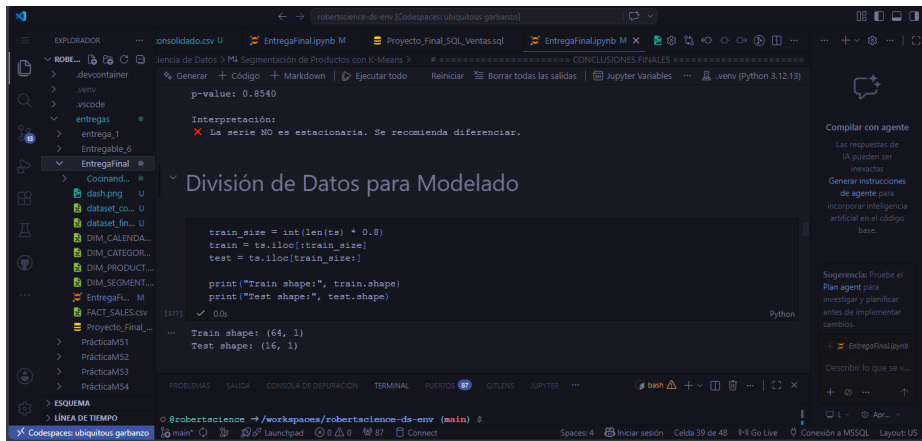
```
ts.set_index("DATE", inplace=True)
ts = ts.rename(columns={"TOTAL_VALUE_SALES": "VENTAS"})

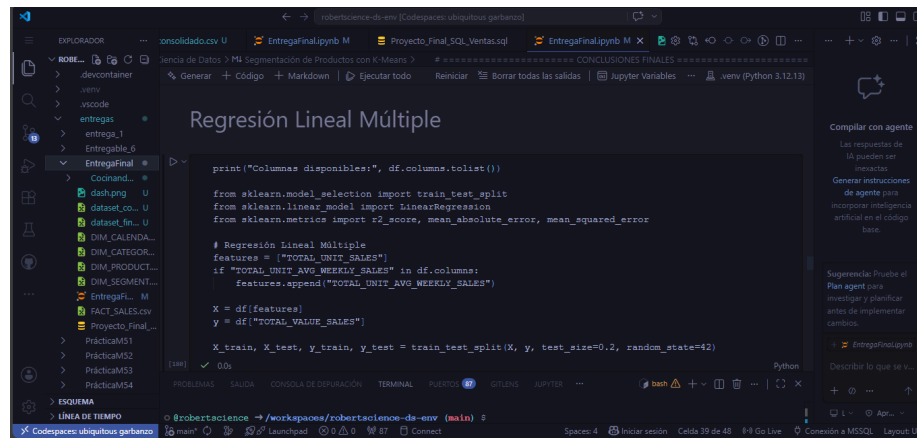
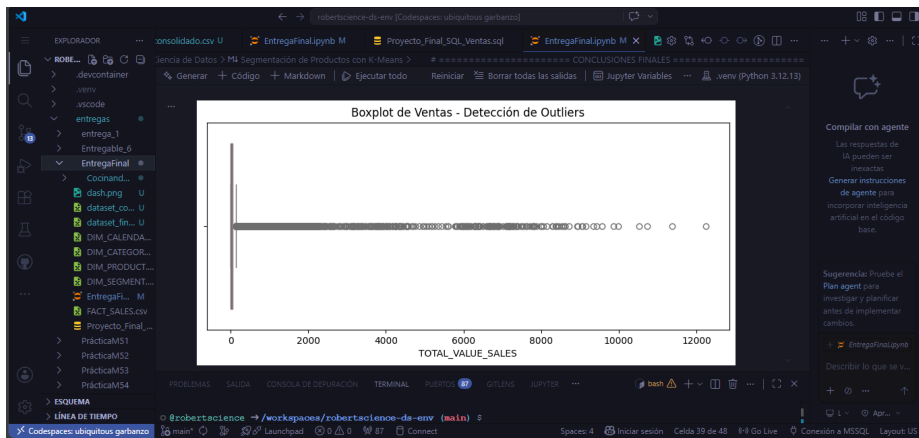
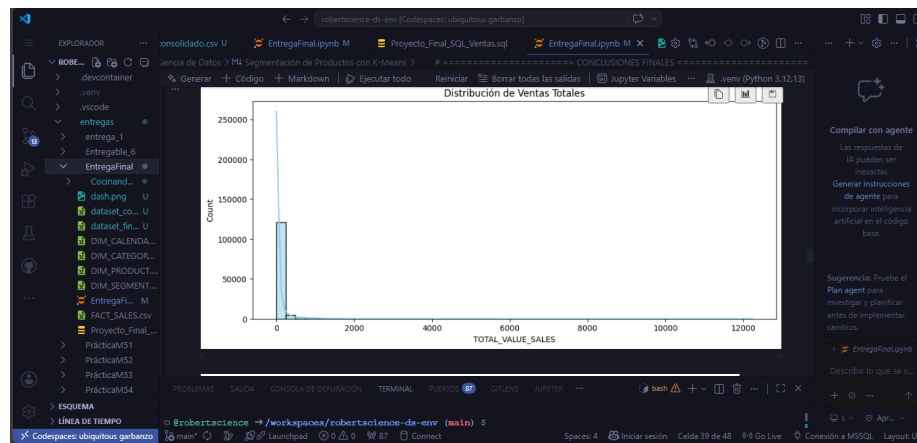
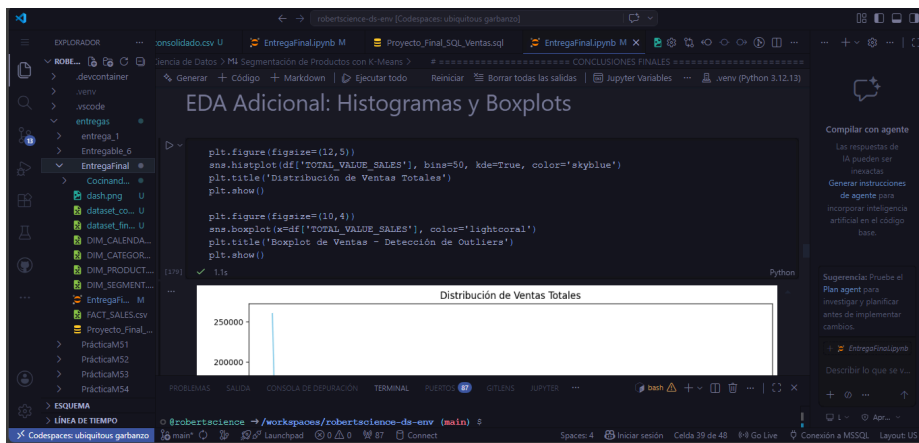
print("\nPrimeras filas de la serie temporal:")
print(ts.head())
print("\nShape de la serie temporal:", ts.shape)
```

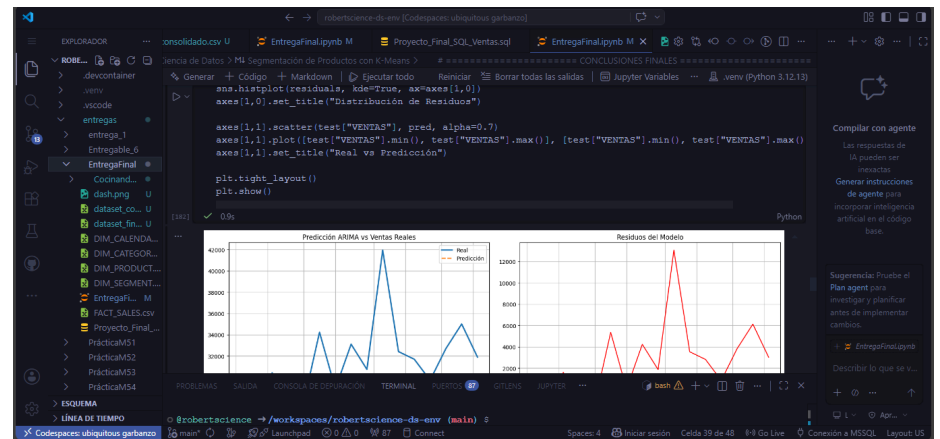
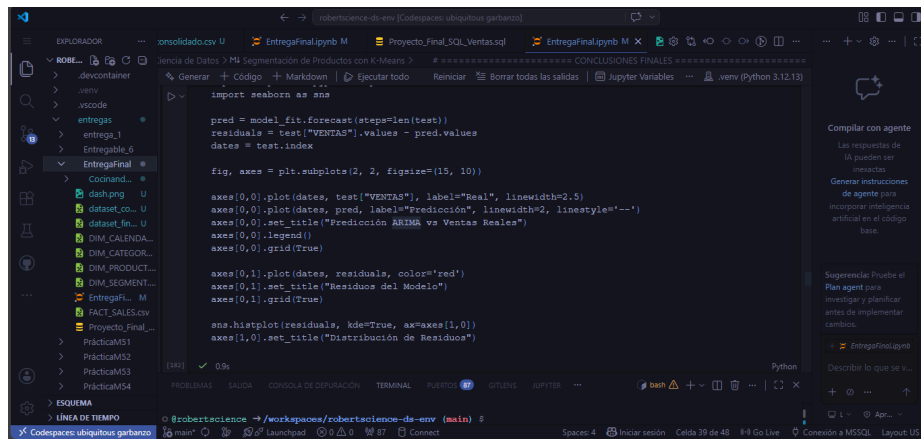
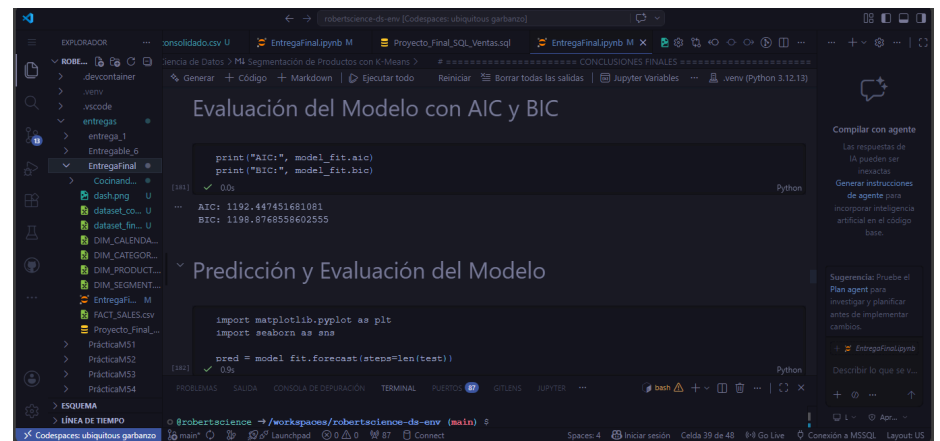
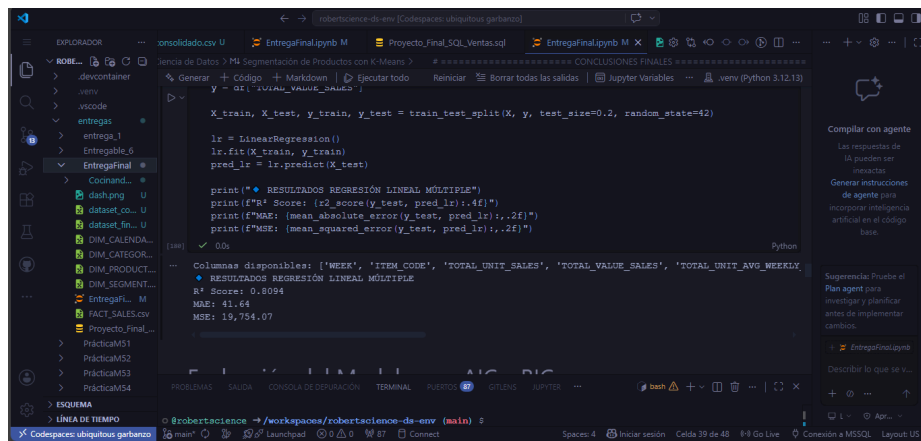
The output shows the first five rows of the time series and its shape:

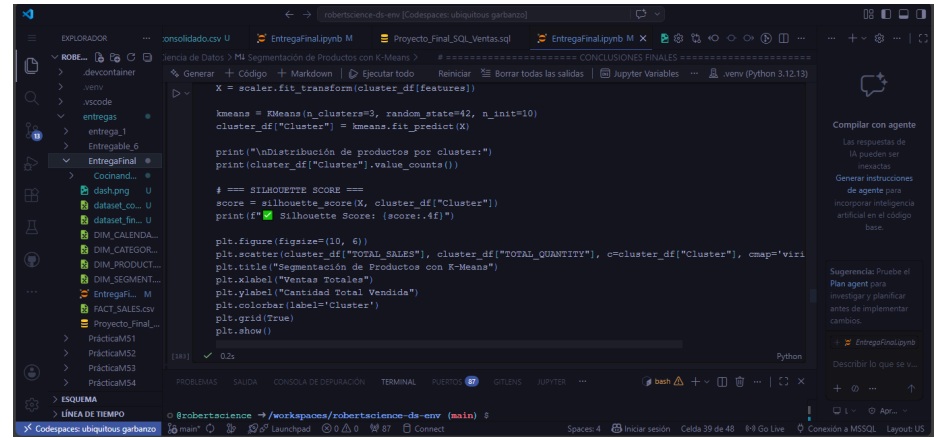
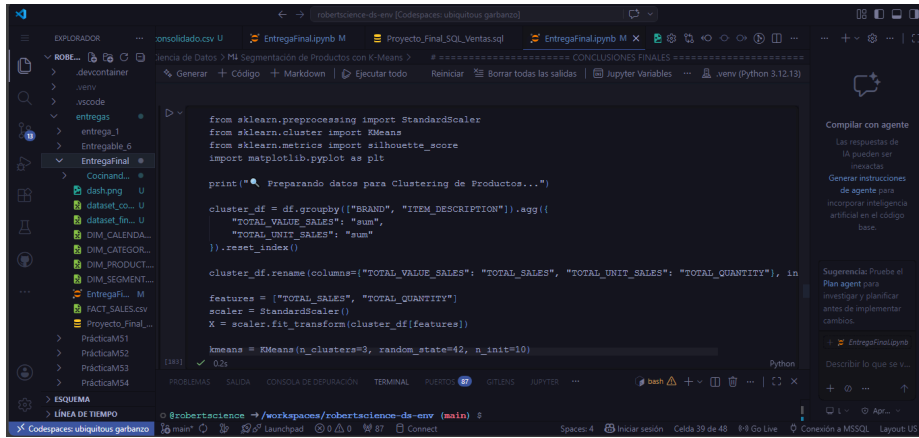
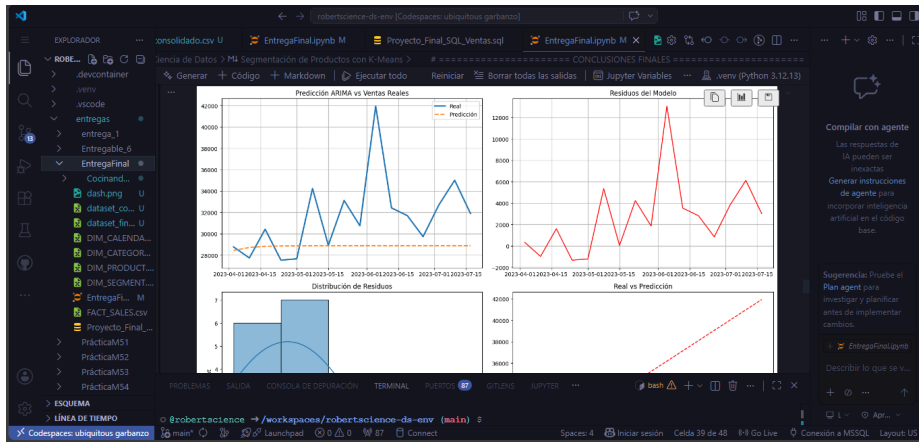
```
Primeras filas de la serie temporal:
VENTAS
DATE
2022-01-09    30991.419
2022-01-16    29996.760
2022-01-23    26003.775
2022-01-30    25970.141
2022-02-06    25910.258

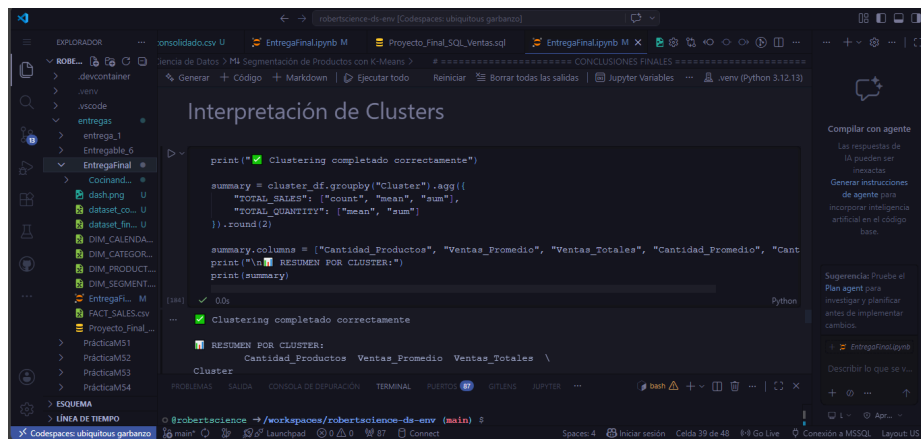
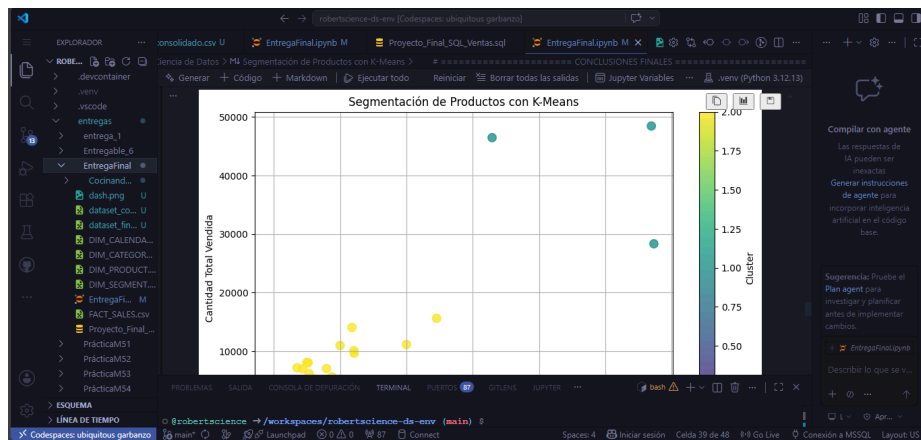
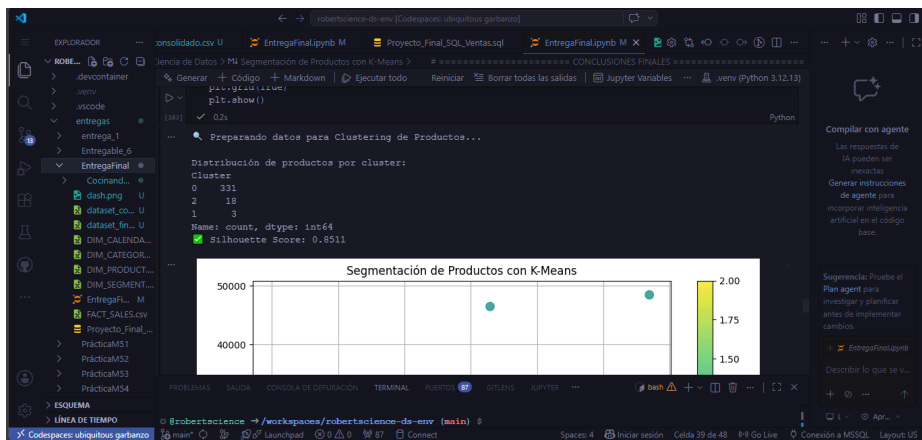
Shape de la serie temporal: (80, 1)
```

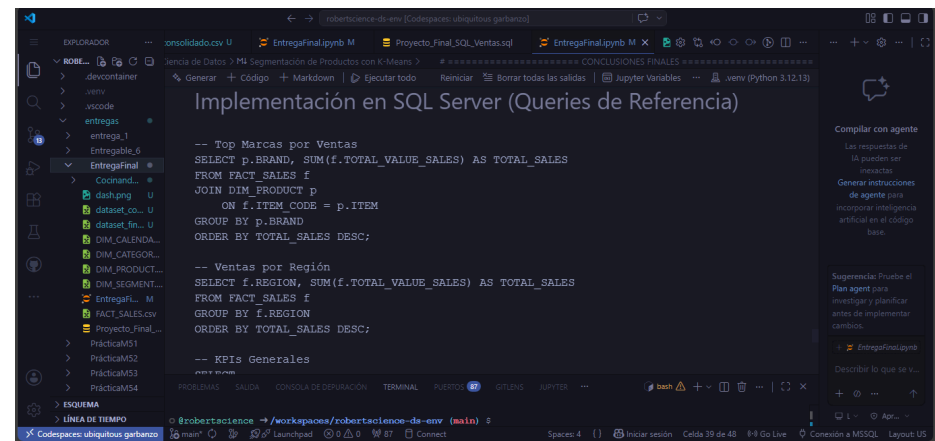













EntregaFinal.pynb M

Proyecto_Final_SQL_Ventas.sql

EntregaFinal.pynb M

CONCLUSIONES FINALES

ORDER BY TOTAL_SALES DBSC;

-- KPIs Generales

SELECT

SUM(TOTAL_VALUE_SALES) AS Total_Ventas,

SUM(TOTAL_UNIT_SALES) AS Total_Unidades,

COUNT(DISTINCT ITEM_CODE) AS Productos_Unicos

FROM FACT_SALES;

Análisis SQL con Dataset Consolidado

En esta sección se simuló el análisis en SQL Server utilizando el dataset consolidado generado previamente en Python. El propósito fue demostrar el conocimiento en consultas analíticas SQL para responder preguntas de negocio relevantes.

Se cargó el archivo consolidado y se realizaron agregaciones, rankings y cálculos de KPIs equivalentes a las consultas que se ejecutarían en un entorno real de SQL Server. Esto complementa el análisis hecho en Python y muestra versatilidad entre ambas herramientas.

Compilar con agente

Las respuestas de IA pueden ser inexactas

Generar instrucciones de agente para incorporar inteligencia artificial en el código base.

Sugerencia: Pruebe el Plan agent para investigar y planificar antes de implementar cambios.

Describa lo que se v...

LINEA DE TIEMPO

Codepaces: ubiquitous garbano

EntregaFinal.pynb M

Proyecto_Final_SQL_Ventas.sql

EntregaFinal.pynb M

CONCLUSIONES FINALES

Se cargó el archivo consolidado y se realizaron agregaciones, rankings y cálculos de KPIs equivalentes a las consultas que se ejecutarían en un entorno real de SQL Server. Esto complementa el análisis hecho en Python y muestra versatilidad entre ambas herramientas.

Dashboard en Power BI

Dashboard Ejecutivo

Se desarrolló un dashboard interactivo con KPIs (Ventas Totales, Unidades, Ticket Promedio), análisis por región/categoría y filtros dinámicos.

Compilar con agente

Las respuestas de IA pueden ser inexactas

Generar instrucciones de agente para incorporar inteligencia artificial en el código base.

Sugerencia: Pruebe el Plan agent para investigar y planificar antes de implementar cambios.

Describa lo que se v...

LINEA DE TIEMPO

Codepaces: ubiquitous garbano

EntregaFinal.pynb M

Proyecto_Final_SQL_Ventas.sql

EntregaFinal.pynb M

CONCLUSIONES FINALES

Dashboard de Análisis de Ventas y Rendimiento Comercial

Proyecto de Ciencia de Datos - Empresa de consumo masivo

38.83K Total Ventas

1.67K Total Unidades

20 Productos

23.24 Ticket Promedio

Total Ventas by CATEGORY

Total Ventas by REGION

Total Ventas by Year

Total Ventas by ITEM_DESCRIPTION

Compilar con agente

Las respuestas de IA pueden ser inexactas

Generar instrucciones de agente para incorporar inteligencia artificial en el código base.

Sugerencia: Pruebe el Plan agent para investigar y planificar antes de implementar cambios.

Describa lo que se v...

LINEA DE TIEMPO

Codepaces: ubiquitous garbano

EntregaFinal.pynb M

Proyecto_Final_SQL_Ventas.sql

EntregaFinal.pynb M

CONCLUSIONES FINALES

Dashboard de Análisis de Ventas y Rendimiento Comercial

Proyecto de Ciencia de Datos - Empresa de consumo masivo

38.83K Total Ventas

1.67K Total Unidades

20 Productos

23.24 Ticket Promedio

Total Ventas by CATEGORY

Total Ventas by REGION

Total Ventas by Year

Total Ventas by ITEM_DESCRIPTION

Compilar con agente

Las respuestas de IA pueden ser inexactas

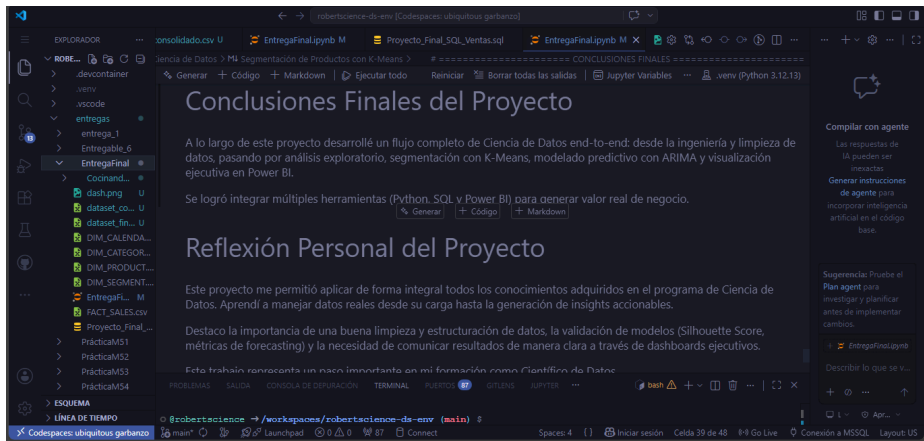
Generar instrucciones de agente para incorporar inteligencia artificial en el código base.

Sugerencia: Pruebe el Plan agent para investigar y planificar antes de implementar cambios.

Describa lo que se v...

LINEA DE TIEMPO

Codepaces: ubiquitous garbano



Conclusiones Finales

Desarrollé un flujo completo de Ciencia de Datos: limpieza y consolidación de datos dimensionales, análisis exploratorio, segmentación con K-Means y modelado predictivo con ARIMA. Los resultados obtenidos permiten generar valor real para la planificación de inventarios y demanda.

Reflexión Personal

Este proyecto me permitió aplicar de forma práctica todos los conocimientos adquiridos, fortaleciendo mis habilidades en el manejo de datos y la generación de insights estratégicos.

Data Analyst & Data Science Solutions

<https://robertscience.online/>

Mayo 2026